

Comunicação entre Processos via Memória Compartilhada Protegida

Daniel Juan , Aldair Ryan, Kaique Hilquias, Joanderson Lima

¹Eixo Tecnológico da Computação – Instituto Federal de Educação, Ciência e Tecnologia do Ceará (IFCE) Campus Maracanaú
Av. Parque Central – 61919-140 – Maracanaú – CE – Brazil

daniel.juan.praxedes07@aluno.ifce.edu.br

Abstract. *Este relatório apresenta os fundamentos da comunicação entre processos (IPC) com ênfase na memória compartilhada (SHM), comparando esse mecanismo com abordagens baseadas em mensagens. Avalia-se por que o acesso concorrente a um segmento comum exige sincronização explícita e como diferentes formas de proteção (mutex, semáforo e travas leitura-escrita) afetam consistência e desempenho. Será mostrado um simulador desenvolvido para observar esses fenômenos de forma visual e reprodutível.*

1. Introdução

Processos independentes precisam trocar informações sem abandonar o isolamento imposto pelo sistema operacional, esse é o papel da *Inter-Process Communication* (IPC).¹ Entre os mecanismos disponíveis, a **memória compartilhada** (SHM) mapeia um mesmo segmento em vários processos, evitando cópias repetidas e sendo adequada quando o estado é consultado e atualizado com frequência.²

Este trabalho trata da **SHM protegida**: o segmento continua sendo o canal de IPC, mas o acesso exige sincronização (mutex, semáforo, RW-lock). Sem regras claras, surgem *race conditions*, bloqueios e impasses. Para observar esses efeitos, foi desenvolvido um simulador em Python com interface web em React, que executa processos concorrentes sobre uma grade que representa o segmento e registra métricas de corrupção, *throughput* e tempo de espera.

2. Fundamentação Teórica

2.1. Isolamento, IPC e memória compartilhada

Cada processo possui espaço de endereço lógico isolado; o SO multiplexa CPU e memória sem permitir que um processo altere arbitrariamente o de outro.³ Quando programas precisam compartilhar estado, o kernel oferece canais de IPC. Além da SHM, foco deste trabalho, há mecanismos por **mensagem** (pipes, filas, sockets), nos quais o SO copia dados entre processos.⁴

¹Andrew S. Tanenbaum e Herbert Bos, *Sistemas Operacionais Modernos*, cap. 2.

²Tanenbaum e Bos, *op. cit.*, cap. 2 e 3.

³Tanenbaum e Bos, *op. cit.*, cap. 3, gerenciamento de memória e espaços de endereço.

⁴Tanenbaum e Bos, *op. cit.*, cap. 2, passagem de mensagens e memória compartilhada.

Tabela 1. Comparativo qualitativo de mecanismos IPC

Critério	Pipe	Socket	Fila msg.	SHM + locks
Cópia explícita	✓	✓	✓	×
Estado compartilhado	×	×	×	✓
Exige sincronização manual	×	parcial	×	✓
Alto throughput leitura	×	×	×	✓ (RW-lock)

Esses canais são simples de raciocinar e isolam bem cada envio, porém custam cópia e transição para o modo kernel; a SHM é preferível quando o mesmo estado é consultado e atualizado com frequência.

Na memória compartilhada, o SO reserva uma região física e a mapeia, via tabelas de páginas, no espaço virtual de dois ou mais processos.⁵ Endereços distintos podem referir a mesma moldura: uma escrita em P1 torna-se visível a P2 sem serializar bytes em mensagem, **IPC por estado compartilhado**. Em UNIX isso aparece em segmentos POSIX (*shmget/shmat*) ou *mmap* compartilhado; em Windows, em *file mapping*.

2.2. Consistência e sincronização

A SHM não garante consistência sozinha. Operações como incrementar um contador envolvem ler, modificar e gravar, etapas intercaláveis entre processos. Se dois leem o mesmo valor e escrevem incrementos separados, perde-se uma atualização: *race condition*.⁶ A proteção vem de primitivas que disciplinam o acesso ao segmento já mapeado:

- **Mutex:** um acesso por vez; evita corrupção, serializa leituras.
- **Semáforo:** até K acessos simultâneos.
- **RW-lock:** vários leitores ou um escritor; favorece cargas com muita leitura.

Mutex, semáforo e RW-lock são mecanismos de sincronização, não de IPC; tornam-se indispensáveis quando o canal é SHM.⁷ O lock pode cobrir o segmento inteiro ou sub-regiões (por célula); granularidade fina aumenta paralelismo, mas favorece *deadlock* se locks forem adquiridos em ordens cruzadas.⁸ No RW-lock global, políticas de *preferência por leitores* ou *justa* alteram o equilíbrio entre throughput de leitura e espera dos escritores.

3. Metodologia e Experimento

Foi implementado um simulador educacional em Python, FastAPI (API REST e WebSoc- ket) e React (interface). O motor mantém uma grade 4×4 que representa o segmento SHM e até 16 processos (em geral 6 por cenário) que, a cada *tick*, leem ou escrevem células com duração configurável. A interface exibe estados (*idle*, *reading*, *writing*, *corrupted*, *waiting*), métricas e log de eventos.

Sem proteção, escritas concorrentes na mesma célula geram corrupção; com locks ativos, a contenção vira fila de espera. Os modos implementados são: *none*, *mutex*

⁵Tanenbaum e Bos, *op. cit.*, cap. 3, compartilhamento de páginas.

⁶Tanenbaum e Bos, *op. cit.*, cap. 2, condições de corrida.

⁷Tanenbaum e Bos, *op. cit.*, cap. 2, semáforos e exclusão mútua.

⁸Tanenbaum e Bos, *op. cit.*, cap. 2, impasses.

global, semáforo (K), RW-lock por célula, RW-lock global (com opção de preferência por leitores ou política justa) e mutex por célula (para demonstrar impasse). Há cenários pré-definidos (corrida de escrita, leituras paralelas, escritor vs. leitores, pool por semáforo, impasse) e modo livre com proteção e número de processos ajustáveis.

As métricas coletadas, corrupções, *throughput*, bloqueios, espera média, deadlocks e paralelismo de leitores, serão usadas no comparativo quantitativo (tabelas e gráficos na etapa seguinte). O protocolo consiste em repetir o mesmo cenário e semente por um número fixo de ticks, alterando apenas a proteção ou parâmetros (K , política do RW-lock). Os processos são simulados no mesmo interpretador Python, não via *shmget*; o modelo reproduz as regras de exclusão e filas, não chamadas de sistema reais.

4. Conclusão

A comunicação entre processos é requisito fundamental dos sistemas multiprogramados. A memória compartilhada destaca-se quando processos consultam e atualizam o mesmo estado com baixa latência; compartilhar endereços, porém, exige sincronização explícita, mutex, semáforo ou RW-lock conforme o padrão de acesso.

O simulador desenvolvido materializa esses trade-offs de forma observável e mensurável. A etapa seguinte consolidará, em figuras e tabelas, o comparativo quantitativo entre modos de proteção no mesmo workload.